Real-Time Detection on American Sign Language Alphabet

Xiaoyu Sun
University of Michigan
xssun@umich.edu

Srikar Varaganti
University of Michigan
vsrikar@umich.edu

Justin Riley Kula
University of Michigan
kulaj@umich.edu

Abstract

In this paper, we tackle the challenge of detecting sign language using computer vision. We accomplished this by training a model on images of the American Sign Language (ASL) alphabet using the YOLOv5 [1] model. We mainly focus on using object detection to recognize the ASL alphabet in real-time.

1. Introduction

Approximately 250,000-500,000 Americans are now using ASL, but qualified interpretation services are not available in many situations where they are needed. The alternative of written communication is cumbersome, impersonal, and even impractical when an emergency occurs. We present an ASL alphabet recognition system that uses YOLOv5 to recognize and translate the user's ASL signs to text in real-time. We choose YOLOv5 because it benefits from the established PyTorch ecosystem: support is simpler, and deployment is easier. We trained the model using an open-source American Sign Language dataset. From a computer vision perspective, this problem represents a significant challenge due to a number of considerations, including:

- Environmental concerns: lighting sensitivity, background, and camera position
- Occlusion: some or all fingers appear, or an entire hand can be out of the field of view
- Sign boundary detection: when a sign ends and the next begins
- Coarticulation: when a sign is affected by a preceding or succeeding sign

After adapting many techniques to improve the accuracy of object detection, including expanding the dataset, data augmentation, hyperparameter evolution, and more, we are able to achieve a relatively accurate ASL alphabet recognition in real-time.

2. Related work

ASL recognition is not a new problem in computer vision. Over the past two decades, researchers have used classifiers from a variety of categories, grouped roughly into linear classifiers, neural networks, and Bayesian networks. While linear classifiers are easy to work with due to their relatively simple models, they require sophisticated feature extraction and preprocessing methods to be successful. Singha and Das [2] obtained an accuracy of 96% on 10 classes for images of one-handed gestures using Karhunen-Loeve Transform. Sharma et al [3] use piece-wise classifiers (Support Vector Machines and k-Nearest Neighbors) to characterize each color channel after background subtraction and noise removal. Their innovation comes from using a contour trace, which is an efficient representation of hand contours. They attain an accuracy of 62.3% using an SVM on the segmented color channel model. Also, some neural networks have been used to tackle ASL translation. Arguably, the most significant advantage of neural networks is that they learn the most important classification features. However, they require considerably more time and data to train. To date, most have been relatively shallow. Mekala et al [4] classified video of ASL letters into text using advanced feature extraction and a 3-layer Neural Network.

In 2012, researchers started to adapt Deep Convolutional Neural networks for image recognition [5]. However, using CNN directly for object detection has many limitations. Later in 2016, researchers came up with an object detection algorithm called Faster R-CNN, which possesses a much shorter training time compared to previous work and significantly improved real-time detection. That same year, Joseph Redmon et al [6] has introduced the YOLO model. YOLO is able to see the entire image during training, which differs from region proposal-based techniques, and improved the background error that Fast R-CNN couldn't handle. In 2020, the YOLO family had grown to version 5; the high accuracy and lightweight of this model defined YOLOv5 as a state-of-the-art object detection model and the leader in real-time object detection.

3. Method

3.1. YOLOv5 Model

For this project, we used the YOLOv5 algorithm that was built completely on PyTorch. The following is a high-level Object detection architecture:

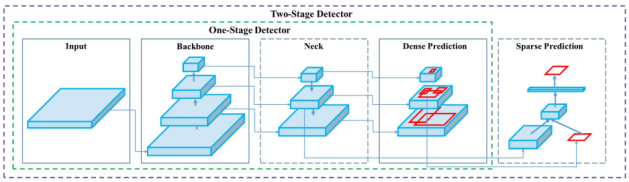

Figure 1. Two stage detector

As shown in Figure 1, the General Object Detector will have a backbone for pre-training and a head to predict classes and bounding boxes. The Backbones can be run on GPU or CPU platforms. The Head can be either one-stage (e.g., YOLO, SSD, RetinaNet) for Dense prediction or two-stage (e.g., Faster R-CNN) for the Sparse prediction object detector. Recent Object detectors have some layers (Neck) to collect feature maps, which are located between the backbone and the Head. Similar to YOLOv4, YOLOv5 has a CSP backbone and a PA-NET neck. The major improvements in YOLOv5 include mosaic data augmentation and auto-learning bounding box anchors. YOLOv5 uses CSPDarknet, which is an open-source neural network written in C and CUDA. Darknet displays information as it loads the configuration file and weights to classify an image and output the top-10 classes for the image. Moreover, the framework can be used to run the neural networks backward in a feature appropriately named “Nightmare”.

3.1.1 Model Architecture

As YOLO v5 is a single-stage object detector, it has three important parts like any other single-stage object detector.

- Model Backbone
- Model Neck
- Model Head

As shown in Figure 2, model Backbone is mainly used to extract significant features from the given input image. In YOLOv5, the CSP-Cross Stage Partial Networks are used as a backbone to extract informative features from an input image. The model Neck is mainly used to generate feature pyramids. Feature pyramids help models to generalize well on object scaling, to identify the same object with different sizes and scales. In YOLOv5, PANet is used as a neck to build feature pyramids. The model Head is mainly used to perform the final detection stage. It applies anchor boxes on features and generates a final output vector with class probabilities, objectness scores, and bounding boxes.

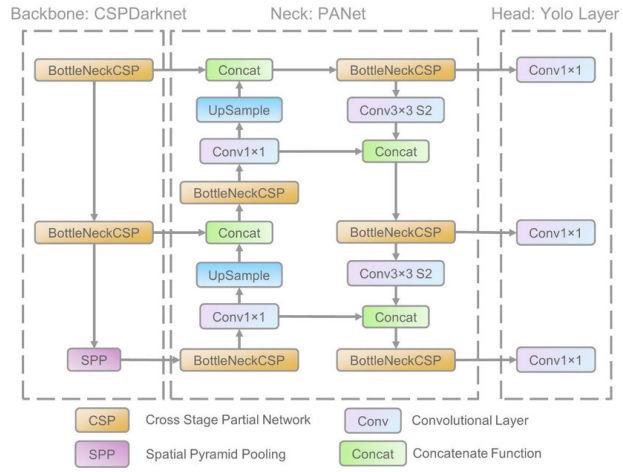

Figure 2. Overall architecture of YOLOv5

3.1.2 Activation Function

The choice of activation functions is crucial in any deep neural network. Recently lots of activation functions have been introduced like Leaky ReLU, mish, swish, etc. The YOLOv5 authors selected the Leaky ReLU and Sigmoid activation function. In YOLOv5 the Leaky ReLU activation function is used in middle/hidden layers and the sigmoid activation function is used in the final detection layer.

3.1.3 Optimization:

In YOLO v5, we have two options for our optimization function:

- SGD
- Adam

The default optimization function for training is SGD. Based on our research, we decided to use Adam optimizer for the final training because Adam works better on small dataset.

3.1.4 Cost function and Loss Function

In the YOLO family, a compound loss is calculated based on objectness score, class probability score, and bounding box regression score.

YOLOv5 uses Binary Cross-Entropy with Logits Loss function from PyTorch for loss calculation of class probability and object score. We used the default loss function used by YOLOv5; since the dataset was not very large, we did not opt for the higher loss function available (Focal loss can also be used with YOLOv5).

3.2. Data Augmentation

We want to build a model that is robust enough to detect American sign language while being less susceptible to background noise, lighting, and the angle by which the user is gesticulating. Data Augmentation encompasses a suite of techniques that enhance the size and quality of training datasets such that better Deep Learning models can be built using them.

To this end, we first adopted horizontal flip to train the model to be able to deal with mirrored perspectives. Secondly, we scale the images up and down to simulate different distances. We have also adopted image translation, which simply involves shifting the image along the X-Y plane. This method of augmentation is very useful, allowing most objects to be located almost anywhere in the image frame. We have also included mosaic augmentation, which is the first new data augmentation technique introduced in YOLOv4. In addition, we have applied image HSV augmentation. Hue augmentation randomly alters the color channels of an input image, causing a model to consider alternative color schemes for objects and scenes in input images. Similarly, saturation augmentation adjusts how vibrant the image is. These techniques are useful to ensure a model is not memorizing a given object or scene's colors.

3.3. Background images

We firstly use an open-source dataset available online, which has images with clean backgrounds for all the alphabet classes. In order to make a robust model that can deal with a complex background, we added 10% background images to the public dataset. Background images are images with no objects that are added to a dataset to reduce False Positives (FP).

3.4. Hyperparameter evolution

We have also leveraged an advanced technique called hyperparameter evolution. Hyperparameter evolution is a method of Hyperparameter Optimization using a Genetic Algorithm (GA) for optimization. Hyperparameters in ML control various aspects of training, and finding optimal values for them can be a challenge. Traditional methods like grid searches can quickly become intractable due to 1) the high dimensional search space 2) unknown correlations among the dimensions, and 3) the expensive nature of evaluating the fitness at each point. These reasons make GA a suitable alternative for hyperparameter searches.

4. Experiment

4.1. Training Data

We used 720 images from a public dataset [7] that was distributed into 26 classes of letters. We have also added 50 background images as introduced in the previous section, which is about 10% of the total training images.

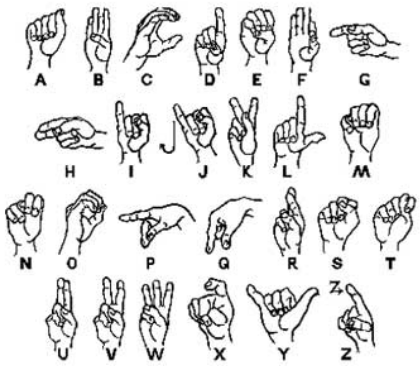

Figure 3. American Sign Language Alphabet

4.2. Metrics

There are various metrics commonly used to measure the accuracy and performance of an object detection model – precision, recall, and mAP. To evaluate the object detection model YOLOv5, the mean average precision (mAP) is used to further tune our model during training. The mAP compares the ground-truth bounding box to the detected box and returns a score. The higher the score, the more accurate the model is in its detections.

4.3. Hyperparameters

To obtain the optimized hyperparameters for training, we used hyperparameter evolution for YOLOv5. We evolved the default hyperparameters that come with yolov5 weights and have successfully improved the accuracy of our model.

4.4. Implementation

Baseline: We initialized the model with YOLOv5s weights, which are pre-trained on the COCO dataset. We observed that the training converged after 80 epochs.

Adding background: In order to reduce the background False Positives, as shown in Figure 5, we added 50 background images with no labels. As shown in Figure 4, we didn't obtain a significant improvement after this addition, but this result was expected since the dataset is still too small.

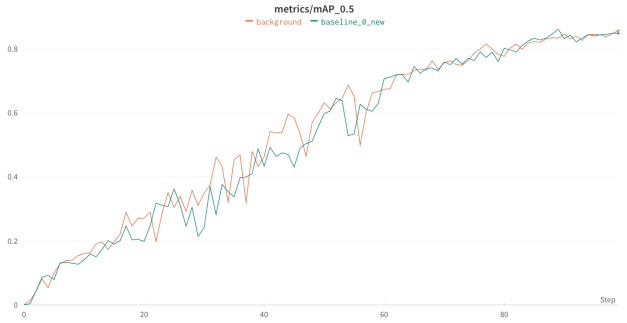

Figure 4. mAP0.5 comparison between baseline and background added training

Evolved hyperparameters: To tune the hyperparameters, we used the hyperparameter evolution technique. A general recommendation is to evolve for a minimum of 300 generations for best results. However, evolution is very expensive and time-consuming, as the base scenario is trained hundreds of times, possibly requiring hundreds or thousands of GPU hours. We evolved for 50 generations to gain improved results within our available computational power. Also, using an upgraded GPU, we have evolved the hyperparameters with YOLOv5m weights, which is a larger structure compared to YOLOv5s. We also changed the default optimizer from SGD to Adam, as some cases showed that Adam works better on small datasets compared to SGD.

Training with evolved hyperparameter: Lastly, we trained the model with the evolved hyperparameter for 150 epochs, as shown in Figure 5 and Figure 6, where we successfully improved our model and outperformed the baseline case.

4.5. Testing

	Baseline	Final version
Ideal test cases	66%	76%
Extreme test cases	15%	32%

We have tested our model on two sets of testing images. Ideal test cases contain images with perfect lighting and upright angle. The extreme test cases are used to verify how robust the model is in imperfect environments. We created the extreme cases based on these three scenarios: 1.) Near the camera and darker image. 2) A bit far and medium brightness. 3.) Too far from the camera and high brightness. Based on the test results we obtained, we can infer that the final model improved substantially from the baseline testing.

5. Conclusions

As introduced in the previous section, we attempted several standard techniques for better training results, and have

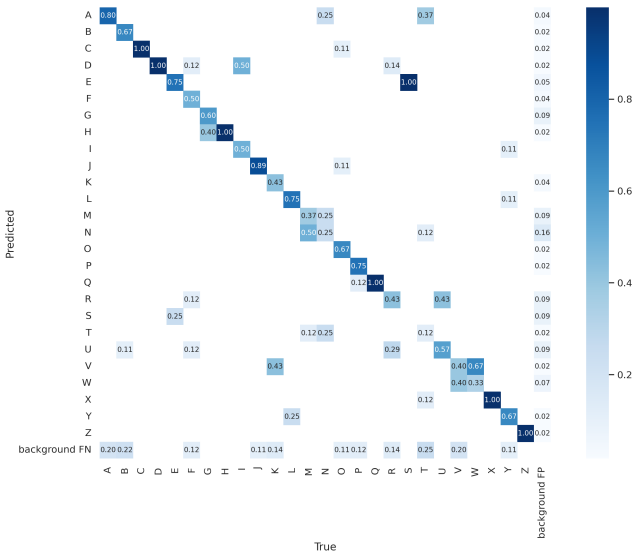

Figure 5. Confusion matrix for baseline training

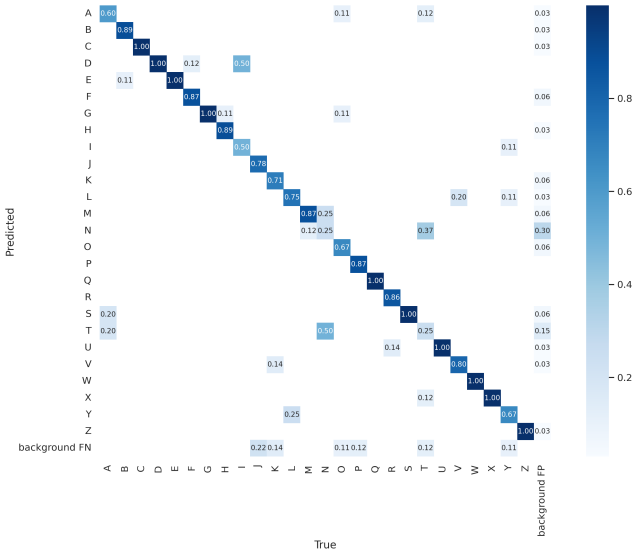

Figure 6. Confusion matrix for final training

successfully doubled the accuracy on the extreme test cases compared to the baseline. However, the accuracy is still relatively low. We can conclude that the training dataset size dominates all the other factors when the set size is too small. Compared to large-scale object detection datasets, our training set is extremely small and results in the insignificant influence of our improvement techniques. In the future, we believe we could have better performance with a larger dataset based on our approach.

References

[1] Glenn Jocher, Alex Stoken, Jirka Borovec, NanoCode012, Ayush Chaurasia, TaoXie, Liu Changyu, Abhiram V, Laughing, tkianai, yxNONG, Adam Hogan, lorenzomammana, AlexWang1900, Jan Hajek, Laurentiu Diaconu, Marc, Yonghye Kwon, oleg, wanghaoyang0106, Yann Defretin, Aditya Lohia, ml5ah, Ben Milanko, Benjamin Fineran, Daniel Khromov, Ding Yiwei, Doug, Durgesh, and Francisco Ingham. ultralytics/yolov5: v5.0 - YOLOv5-P6 1280 models, AWS, Supervise.ly and YouTube integrations, April 2021.

[2] Joyeeta Singha and Karen Das. Hand gesture recognition based on karhunen-loeve transform. *CoRR*, abs/1306.2599, 2013.

[3] Rohit Sharma, Yash Nemani, Sumit Kumar, Lalit Kane, and Pritee Khanna. Recognition of single handed sign language gestures using contour tracing descriptor.

[4] Priyanka Mekala, Ying Gao, Jeffrey Fan, and Asad Davari. Real-time sign language recognition based on neural network architecture. In *2011 IEEE 43rd Southeastern Symposium on System Theory*, pages 195–199, 2011.

[5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60:84 – 90, 2012.

[6] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2016.

[7] David Lee. American sign language letters dataset, 2020.